

Arquitectura final CLOUDAPP

Vista general

```
Azure Static Web Apps (React)
|
v
Azure Container Apps (FastAPI)
|
+--> Azure PostgreSQL cloudapp
|     - datos operativos de la app
|     - auditoria, gobierno, catalogo, DataOps, Sentinel storage
|
+--> Azure PostgreSQL lab
|     - base bancaria simulada
|     - pg_stat_activity, pg_locks, pg_stat_database, pg_stat_statements
|
+--> OpenAI opcional
+--> Databricks opcional
+--> DataHub/Purview opcional
```

CLOUDAPP es una plataforma demo de gobierno, operaciones y observabilidad de datos. El frontend muestra el estado de plataforma, Query Governance, DBA Copilot, DataOps Monitor, Catalog Governance, Autopilot y DB Sentinel AI. El backend centraliza reglas, auditoria, metadatos, predicciones, RCA e integraciones externas.

Bases de datos

cloudapp es la base de la aplicacion. Guarda usuarios demo, auditoria, politicas de Query Governance, perfiles DBA, catalogo interno, ejecuciones DataOps, reportes Autopilot, muestras de Sentinel e incidentes.

lab es la base monitoreada por Sentinel. Contiene el simulador bancario y las vistas de telemetria PostgreSQL. Sentinel no guarda resultados ahi; solo lee metricas y slow query fingerprints para copiarlas a **cloudapp**.

Para crear/cargar **lab** desde el repo:

```
.\scripts\azure-lab-bootstrap.ps1 -Password "<POSTGRES_PASSWORD>"
```

Modos Sentinel

Local Lab Mode usa Docker/PostgreSQL local para ensayos sin tocar Azure. Sirve para desarrollo, pruebas de scripts y fallos controlados.

Azure Demo Mode usa Azure PostgreSQL **lab**. Sirve para la demo desplegada: el backend en Container Apps recolecta metricas reales desde **SENTINEL_MONITOR_DB_URL**.

Variables importantes:

```
SENTINEL_MONITOR_DB_URL=postgresql+psycopg://...@psql-cloudapp-dev-  
zgc5ku4.postgres.database.azure.com:5432/lab?sslmode=require  
SENTINEL_MONITOR_DATABASE_NAME=lab  
SENTINEL_ENABLE_AUTO_COLLECT=true  
SENTINEL_COLLECT_INTERVAL_SECONDS=60  
SENTINEL_RISK_THRESHOLD=0.70
```

Flujo DB Sentinel AI

```
Simulation Lab -> collector -> sentinel_metric_samples  
                    -> sentinel_query_samples  
                    -> predictor -> RCA -> Copilot  
                    -> sentinel_incidents si hay riesgo real
```

El dashboard ahora muestra estado de monitor DB, modo local/Azure, ultima muestra, total de muestras, auto collect, predictor y RCA. El flujo controlado prepara una simulacion, fuerza una recoleccion, predice riesgo y persiste incidente solo si el riesgo supera el umbral o el modelo predice incidente.

Gobierno y catalogo

Query Governance expone inventario por base, schema, tablas y columnas. Tambien marca que tablas son queryables y cuales son internas.

DBA Copilot muestra las fuentes analizadas para separar **cloudapp** de **lab**.

Catalog Governance diferencia origen por **source_system**, **platform**, **database_name**, **schema_name**, **table_name** y **layer**. Asi puedes explicar si un asset viene de PostgreSQL, DataOps/Databricks o un catalogo externo.

Despliegue

El contenedor ejecuta:

```
alembic upgrade head  
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --log-level info
```

GitHub Actions construye la imagen, actualiza secretos de OpenAI, Databricks y Sentinel, y aplica variables de entorno al Container App. Terraform tambien queda preparado para inyectar Sentinel si **sentinel_monitor_db_url** se define.